

# School Management App

## Laravel Sanctum Authentikáció & Jogosultságkezelés

Laravel 12 + Clean Code elvek | 2026

### 1. Mi az autentikáció és miért kell?

Az autentikáció (hitelesítés) az a folyamat amellyel egy rendszer megállapítja ki a felhasználó – bejelentkezés útján. Ez különbözik az autorizációtól (jogosultságkezelés), ami azt dönti el mit tehet meg a már azonosított felhasználó.

#### 1.1 Authentikáció vs Autorizáció

| Fogalom       | Kérdés amire válaszol | Projektben                                         |
|---------------|-----------------------|----------------------------------------------------|
| Authentikáció | Ki vagy te?           | Sanctum token ellenőrzés (auth:sanctum middleware) |
| Autorizáció   | Mit tehetsz meg?      | role middleware + Form Request authorize()         |

### 2. Laravel Sanctum – Token alapú autentikáció

A Sanctum egy egyszerű token alapú API autentikációs rendszer. A hagyományos session-alapú autentikációval szemben (ami cookie-kat használ) a Sanctum egy tokent ad, amit a kliens minden kéréshez csatol.

#### 2.1 A teljes bejelentkezési folyamat

```
1. Frontend elküldi: POST /api/auth/login
   { "email": "admin@school.com", "password":
"admin123" }

2. Laravel ellenőrzi az email + jelszó párt az adatbázisban

3. Ha helyes → generál egy tokent és visszaküldi:
   {
     "token": "1|xsirbGB2V2AeZrnVBAMQHd1...",
     "token_type": "Bearer",
```

```

        "user": { "id": 1, "name": "Admin", "role":
"admin" }
    }

4. Frontend elmenti a tokenet (pl. localStorage)

5. Minden további kérésnél csatolja:
    Authorization: Bearer 1|xsirbGB2V2AeZrnVBAMQHd1...

6. A Sanctum middleware minden védett route-on ellenőrzi a tokenet
    → Ha érvényes: folytatódik a kérés a Controllerhez
    → Ha nem: 401 Unauthorized

```

### Mi az a Bearer token?

A 'Bearer' szó azt jelenti: 'aki felmutatja, jogosult'. A token olyan mint egy belépőkártya – nem kell újra bejelentkezni minden kéréshez, elég a tokenet csatolni. Fontos hogy HTTPS-en közlekedjen, mert a token felmutatásával bárki hozzáférhet a fiókhoz.

## 2.2 A token felépítése

```

1|xsirbGB2V2AeZrnVBAMQHd1kaWvwhiIk6g6h49vef0dcf886
| ────────────────────────────────────────────────────┘
|                                     a token titkos része (hash-elve tárolva)
└─ a personal_access_tokens tábla rekord id-ja

```

### Hol tárolódik a token?

A Sanctum egy külön táblát hoz létre: `personal_access_tokens`. Ide kerül minden generált token hash-elt formában (nem plain textként!). Amikor a frontend elküldi a tokenet, a Sanctum ezt a hash-t hasonlítja össze – hasonlóan ahogy a jelszavaknál működik a `Hash::check()`.

## 3. AuthService – a bejelentkezési logika

Az AuthService kezeli a bejelentkezés, kijelentkezés üzleti logikáját – ugyanúgy ahogy a többi Service teszi a saját domainjében.

### 3.1 login() – lépésről lépésre

```

public function login(string $email, string $password): array
{
    // 1. Megkeressük a felhasználót email alapján

```

```

$user = User::where('email', $email)->first();

// 2. Ellenőrizzük hogy létezik-e ÉS helyes-e a jelszó
if (!$user || !Hash::check($password, $user->password)) {
    throw ValidationException::withMessages([
        'email' => ['Érvénytelen email cím vagy jelszó.'],
    ]);
}

// 3. Új token generálása
$token = $user->createToken('auth_token')->plainTextToken;

return [
    'token'          => $token,
    'token_type'     => 'Bearer',
    'user'           => [...],
];
}

```

### Miért nem áruljuk el hogy az email vagy a jelszó hibás?

Biztonsági okból. Ha azt mondanánk 'helytelen jelszó', a támadó tudná hogy az email cím létezik a rendszerben – ez egy támadási felület (user enumeration). Az 'Érvénytelen email cím vagy jelszó' üzenet mindkét esetben ugyanaz, így a támadó nem tud következtetni semmire.

### Miért Hash::check() és nem Auth::attempt()?

Az Auth::attempt() a session-alapú (webes) autentikációhoz készült – cookie-t állít be. API-nál nekünk saját magunknak kell ellenőrizni a jelszót a Hash::check()-kel, majd manuálisan generálni a token. A Hash::check() a megadott jelszót a tárolt bcrypt hash-hez hasonlítja – nem lehet visszafejteni, csak összehasonlítani.

### A plainTextToken csak EGYSZER látható!

A createToken() visszaadja a token teljes szövegét (plainTextToken), de ez csak ABBAN a válaszban jelenik meg amikor létrejön. Az adatbázisban már csak a hash-elt verzió van tárolva. Ha a felhasználó elveszíti a token, nem lehet 'visszakérni' – újra be kell jelentkeznie.

## 3.2 logout() vs logout-all()

```

// Kijelentkezés csak az AKTUÁLIS eszközről
public function logout(User $user): void
{
    $user->currentAccessToken()->delete();
}

// Kijelentkezés MINDEN eszközről

```

```
public function logoutAll(User $user): void
{
    $user->tokens()->delete();
}
```

| Metódus              | Mit töröl                       | Mikor használjuk                    |
|----------------------|---------------------------------|-------------------------------------|
| currentAccessToken() | Csak a jelenlegi kérés tokenjét | Normál kijelentkezés                |
| tokens()             | Az összes token (minden eszköz) | Gyanús aktivitás, jelszó csere után |

## 4. Szerepkörök (Roles) – admin, teacher, parent

A rendszerben három szerepkör létezik. A users tábla role mezője egy enum – ez garantálja hogy csak ez a három érték kerülhet bele.

### 4.1 A migráció

```
Schema::table('users', function (Blueprint $table) {
    $table->enum('role', ['admin', 'teacher', 'parent'])
        ->default('admin')
        ->after('email');
});
```

#### Miért enum és nem string?

Az enum adattípus adatbázis szinten garantálja hogy csak a felsorolt értékek kerülhetnek a mezőbe. Ha valaki véletlenül 'Admin' (nagybetűvel) vagy 'administrator'-t próbálna beszúrni, az adatbázis elutasítaná. Ez egy védelmi réteg már a legalsó szinten.

### 4.2 User modell – szerepkör ellenőrző metódusok

```
public function isAdmin(): bool
{
    return $this->role === 'admin';
}

public function isTeacher(): bool
{
    return $this->role === 'teacher';
}
```

```

public function isParent(): bool
{
    return $this->role === 'parent';
}

// Több szerepkör ellenőrzése egyszerre
public function hasRole(array $roles): bool
{
    return in_array($this->role, $roles);
}

// Használat:
auth()->user()->isAdmin(); // true/false
auth()->user()->hasRole(['admin', 'teacher']); // true ha bármelyik

```

### Miért ezek a metódusok és nem közvetlen összehasonlítás?

A `$user->role === 'admin'` mindenhol működne, de a `isAdmin()` metódus olvashatóbb és karbantarthatóbb. Ha a jövőben megváltozna a logika (pl. 'super\_admin' szerepkör bevezetése, ami szintén admin jogokkal rendelkezik), csak EZEN a helyen kell módosítani – nem az egész kódrendszerben keresgélni minden `=== 'admin'` összehasonlítást.

## 5. Egyedi Middleware – EnsureUserHasRole

A Laravel nem tartalmaz alaphoz role-alapú middleware-t, ezért létrehoztunk egy sajátot. Ez a middleware route szinten véd egész végpont csoportokat.

### 5.1 A middleware kódja

```

class EnsureUserHasRole
{
    public function handle(Request $request, Closure $next, string
...$roles): Response
    {
        $user = $request->user();

        if (!$user->hasRole($roles)) {
            return response()->json([
                'message' => 'Nincs jogosultságod ehhez a művelethez.'
            ], 403);
        }

        return $next($request);
    }
}

```

### Mi az a string ...\$roles variadic paraméter?

A ... (spread/variadic operátor) lehetővé teszi hogy a metódus tetszőleges számú paramétert fogadjon. Amikor a route-ban ezt írjuk: `middleware('role:admin,teacher')`, a Laravel automatikusan szétbontja vesszőknél és átadja mint `$roles = ['admin', 'teacher']`. Így egyetlen middleware osztály kezeli az összes szerepkör-kombinációt.

### Miért nincs itt bejelentkezés-ellenőrzés?

Mert ez a middleware MINDIG az `auth:sanctum` UTÁN fut a route definícióban (`Route::middleware(['auth:sanctum', 'role:admin'])`). Ha idáig eljutott a kérés, a felhasználó már biztosan be van jelentkezve – az `auth:sanctum` middleware már ellenőrizte és 401-et adott volna vissza ha nem.

## 5.2 Middleware regisztrálása – bootstrap/app.php

```
->withMiddleware(function (Middleware $middleware) {  
    $middleware->alias([  
        'role' => \App\Http\Middleware\EnsureUserHasRole::class,  
    ]);  
});
```

### Mi az az alias?

Az alias egy rövidebb nevet ad a middleware osztálynak. Enélkül minden route-ban ki kellene írni a teljes namespace-t:  
`middleware(\App\Http\Middleware\EnsureUserHasRole::class . ':admin')`. Az alias-szal egyszerűen: `middleware('role:admin')`. Sokkal olvashatóbb.

## 5.3 Middleware használata route-okon

```
// Csak admin érheti el - teljes CRUD  
Route::middleware('role:admin')->group(function () {  
    Route::apiResource('teachers', TeacherController::class)  
        ->only(['store', 'update', 'destroy']);  
});  
  
// Admin ÉS teacher is elérheti  
Route::middleware('role:admin,teacher')->group(function () {  
    Route::post('attendances/students', [AttendanceController::class,  
        'storeStudents']);  
});  
  
// Mindenki elérheti (csak auth:sanctum kell, role nem)  
Route::apiResource('teachers', TeacherController::class)  
    ->only(['index', 'show']);
```

## 6. Kétszintű védelem – Middleware + Form Request

A projektben a védelmi rendszer két szinten dolgozik, amelyek kiegészítik egymást. Ez a legprofesszionálisabb megközelítés API fejlesztésben.

### 6.1 A két szint különbsége

| Szint              | Kérdés                                    | Példa                           | Hol van a kódban    |
|--------------------|-------------------------------------------|---------------------------------|---------------------|
| Middleware (route) | Egyáltalán beléphet-e ide ez a szerepkör? | Csak admin hozhat létre tanárt  | routes/api.php      |
| Form Request       | Ezt a KONKRÉT erőforrást módosíthatja-e?  | A tanár csak a saját tantárgyát | authorize() metódus |

#### Miért nem elég csak a middleware?

A middleware csak DURVA szűrést tud végezni – 'ez a szerepkör beléphet-e ide'. De nem tudja megnézni a konkrét adatokat (pl. melyik vizsgáról van szó, ki a tantárgy tanára). A Form Request authorize() metódusa hozzáfér a route paraméterekhez és az adatbázishoz is – itt lehet FINOM, adat-specifikus szabályokat írni.

### 6.2 Példa – StoreExamResultRequest

```
public function authorize(): bool
{
    $user = $this->user();

    // Admin bármilyen vizsgálathoz rögzíthet eredményt
    if ($user->isAdmin()) {
        return true;
    }

    if ($user->isTeacher()) {
        $exam = Exam::find($this->route('id'));

        if (!$exam) {
            return false;
        }

        // Csak a SAJÁT tantárgyához tartozó vizsgát módosíthatja
        return $user->teacher?->id === $exam->subject->teacher_id;
    }
}
```

```
    return false;
}
```

### Miért find() és nem findOrFail()?

Az authorize() metódus nem dobhat 404-es hibát – csak true/false-t adhat vissza. Ha findOrFail()-t használnánk és nem találná a vizsgát, egy nem kezelt kivétel keletkezne. A find() null-t ad vissza ha nincs találat, amit egyszerűen ellenőrizhetünk és false-t adhatunk vissza (ami 403-as választ generál).

### A nullsafe operátor (?->) szerepe itt

\$user->teacher->id – ha a bejelentkezett tanárnak valamiért (hibásan) nincs Teacher rekordja hozzárendelve, a \$user->teacher null lenne. A ?-> biztonságosan null-t ad vissza ahelyett hogy hibát dobna, és a null !== \$exam->subject->teacher\_id összehasonlítás egyszerűen false lesz – vagyis nincs jogosultság.

## 7. User ↔ Teacher/Student/Guardian összekapcsolás

A rendszerben eredetileg a User (bejelentkezési fiók) és a Teacher/Student/Guardian (iskolai adat) két teljesen külön tábla volt kapcsolat nélkül. Ez megakadályozta hogy tudjuk melyik bejelentkezett tanárhoz melyik iskolai Teacher rekord tartozik.

### 7.1 A probléma

```
// Bejelentkezik egy 'teacher' szerepkörű User
auth()->user(); // User { id: 2, role: 'teacher' }

// De honnan tudjuk melyik Teacher rekord az övé?
// A teachers táblában nincs semmi ami a User(id:2)-re mutatna!
Teacher::where('???', 2)->first(); // Nem lehet lekérdezni
```

### 7.2 A megoldás – user\_id foreign key

```
// Migráció – user_id hozzáadása a teachers táblához
Schema::table('teachers', function (Blueprint $table) {
    $table->foreignId('user_id')
        ->nullable()
        ->unique()
        ->after('id')
        ->constrained('users')
        ->onDelete('set null');
});
```



| Tulajdonság | Miért kell?                                                                 |
|-------------|-----------------------------------------------------------------------------|
| nullable()  | Nem minden tanárnak van (még) bejelentkezési fiókja                         |
| unique()    | Egy User fiók csak EGY Teacher rekordhoz tartozhat                          |
| set null    | Ha a User törlődik, a Teacher rekord megmarad (csak a kapcsolat szűnik meg) |

## 7.3 A kapcsolatok iránya

```
// Teacher modellben - belongsTo, mert a teachers tábla
// tartalmazza a user_id foreign key-t
public function user(): BelongsTo
{
    return $this->belongsTo(User::class);
}

// User modellben - hasOne, mert egy Userhez pontosan
// EGY Teacher rekord tartozhat (a unique() megkötés miatt)
public function teacher(): HasOne
{
    return $this->hasOne(Teacher::class);
}

public function student(): HasOne
{
    return $this->hasOne(Student::class);
}

public function guardian(): HasOne
{
    return $this->hasOne(Guardian::class, 'user_id');
}
```

### belongsTo vs hasOne – hogyan döntjük el?

Ökölszabály: amelyik táblában a foreign key TALÁLHATÓ, az a modell kap belongsTo-t. A teachers táblában van user\_id, ezért Teacher::user() belongsTo. A users táblában NINCS teacher\_id, csak a teachers oldaláról hivatkoznak rá – ezért User::teacher() hasOne (nem hasMany, mert unique()).

## 7.4 Használat gyakorlatban

```
// Bejelentkezett tanár saját adatainak elérése:
$user = auth()->user();
$user->teacher; // Teacher modell objektum
```

```

$user->teacher->id;          // A Teacher rekord id-ja
$user->teacher->subjects;    // A tanár tantárgyai (Teacher kapcsolatai)

// Ellenőrzés hogy a tanár jogosult-e egy adott tantárgyhoz:
$exam->subject->teacher_id == $user->teacher->id;

```

## 8. AuthController – me() metódus dinamikus kapcsolat betöltéssel

```

public function me(Request $request): UserResource
{
    $user = $request->user();

    // Csak a releváns kapcsolatot töltjük be a szerepkör alapján
    $user->load(match(true) {
        $user->isTeacher() => 'teacher',
        $user->isParent()  => 'guardian',
        default            => [],
    });

    return new UserResource($user);
}

```

### Miért csak a releváns kapcsolatot töltjük be?

Egy admin usernek nincs teacher/student/guardian rekordja – felesleges lenne mindhármat lekérdezni minden bejelentkezéskor. A `match(true)` kifejezéssel pontosan eldöntjük melyik kapcsolatra van szükség a szerepkör alapján, és csak azt az egyet töltjük be a `load()` metódussal. Ez egy kis, de fontos teljesítmény-optimalizáció.

### A `load()` vs `with()` különbsége

A `with()` egy lekérdezés ELŐTT használatos (`Model::with(['x'])->get()`). A `load()` egy MÁR LÉTEZŐ objektumon tölt be utólag egy kapcsolatot (`$user->load('teacher')`). Itt azért kell `load()`, mert a `$user` objektum már létezik (a `request->user()`-ből jött) – nem egy friss lekérdezésről van szó.

## 9. UserResource – szerepkör-specifikus JSON válasz

```

'teacher' => $this->whenLoaded('teacher', function () {
    return $this->teacher ? [
        'id'           => $this->teacher->id,
        'name'         => $this->teacher->name,
    ] : [];
});

```

```
    'qualification' => $this->teacher->qualification,  
    ] : null;  
  }},
```

### whenLoaded() itt is N+1 védelem

Ugyanaz az elv mint a többi Resource-nál: ha a Controller (AuthController::me()) nem töltötte be a teacher kapcsolatot (mert pl. admin a felhasználó), ez a mező egyszerűen nem jelenik meg a JSON válaszban – nem próbál új lekérdezést indítani.

## 10. A teljes védelmi lánc – összefoglaló diagram

Kérés: PUT /api/exams/5/results (tanár tokennel)

1. Middleware: auth:sanctum  
→ Érvényes-e a token? → IGEN, folytatódik
2. Middleware: role:admin,teacher  
→ A felhasználó admin vagy teacher? → IGEN (teacher), folytatódik
3. Form Request: StoreExamResultRequest  
→ authorize(): a tanár SAJÁT tantárgyához tartozik a vizsga?  
→ HA IGEN: folytatódik a rules() validációhoz  
→ HA NEM: 403 Forbidden – itt áll meg!
4. rules(): validáció (marks\_obtained szám, exists stb.)  
→ HA hibás: 422 Unprocessable Entity
5. Controller: storeResults()  
→ Service hívás, üzleti logika, válasz

## 11. UserSeeder – teszt felhasználók valódi kapcsolatokkal

A Seeder nem csak User rekordokat hoz létre, hanem ténylegesen összeköti őket a Teacher/Guardian rekordokkal a user\_id mezőn keresztül.

```
$teacherUser = User::updateOrCreate(  
    ['email' => 'teacher@school.com'],  
    ['name' => 'Kovács Tanár', 'password' => Hash::make('teacher123'),  
    'role' => 'teacher']  
);
```

```
Teacher::updateOrCreate(
    ['email' => 'teacher@school.com'],
    [
        'user_id'      => $teacherUser->id,    // ← itt köti össze!
        'name'         => 'Kovács Tanár',
        'qualification' => 'Matematika-Fizika szakos tanár',
    ]
);
```

### Miért updateOrCreate() és nem create()?

A Seeder többször is lefuttatható (pl. migrate:fresh --seed közben) – az updateOrCreate() nem hoz létre duplikátumokat, hanem frissíti a meglévő rekordot ha már létezik az email alapján. Ez idempotens működést biztosít: a Seeder akárhányszor lefuttatva ugyanazt az eredményt adja.

## 12. Összefoglaló – Auth rendszer komponensei

| Komponens                                             | Felelősség                                      |
|-------------------------------------------------------|-------------------------------------------------|
| users.role (enum)                                     | Szerepkör tárolása: admin, teacher, parent      |
| User::isAdmin()/isTeacher()/isParent()                | Szerepkör ellenőrzés olvasható formában         |
| AuthService                                           | Bejelentkezés, kijelentkezés üzleti logikája    |
| AuthController                                        | HTTP kérés/válasz kezelése az auth végpontokhoz |
| EnsureUserHasRole middleware                          | Route szintű durva szerepkör-szűrés             |
| Form Request authorize()                              | Kérés szintű finom, adat-specifikus jogosultság |
| teachers.user_id / students.user_id / parents.user_id | User ↔ iskolai adat összekapcsolás              |
| UserResource                                          | Szerepkör-specifikus JSON válasz (whenLoaded)   |
| UserSeeder                                            | Teszt felhasználók valódi kapcsolatokkal        |

### Auth mappa struktúra

```
app/
├── Http/
│   ├── Controllers/Api/
│   │   └── AuthController.php      ← login, logout, logout-all, me
│   ├── Middleware/
│   │   └── EnsureUserHasRole.php  ← role:admin,teacher middleware
│   └── Resources/
```

```

|       └─ UserResource.php           ← szerepkör-specifikus válasz
└─ Models/
  └─ User.php                         ← isAdmin(), hasRole(),
teacher() / student() / guardian()
  └─ Teacher.php                     ← user() belongsTo
  └─ Student.php                     ← user() belongsTo
  └─ Guardian.php                    ← user() belongsTo
└─ Services/
  └─ AuthService.php                  ← login(), logout(), logoutAll()

database/
└─ migrations/
  └─ ..._add_role_to_users_table.php
  └─ ..._add_user_id_to_teachers_table.php
  └─ ..._add_user_id_to_students_table.php
  └─ ..._add_user_id_to_parents_table.php
└─ seeders/
  └─ UserSeeder.php                  ← teszt felhasználók + kapcsolatok

bootstrap/
└─ app.php                          ← middleware alias regisztráció

```

– School Management App fejlesztési dokumentáció –